

1 Part 1: Data Structures in R

2 Summary

3 This Part begins our exploration of basic concepts of the **R programming**
4 **language**.

5 The development here follows closely that of the textbook *Advanced R* by
6 Hadley Wickham, CRC Press, 2014.

7 It is assumed that you are somewhat familiar with basic programming con-
8 cepts, and that you have installed **R** and **RStudio** on your computer.

1 Getting Help

2 The most important function in R is `help()`. For any function described
3 below, there are almost always additional arguments that can be specified
4 to extend the usefulness of the function.

5 For example, looking at `help(mean)` tells the user how to adjust the re-
6 sult to calculate the trimmed mean, and how to have the function handle
7 missing values.

8 See part of the output on the following slide.

9 In addition, the webpage <http://Rseek.org> is a useful Google-like resource
10 for finding information regarding R.

Description

Generic function for the (trimmed) arithmetic mean.

Usage

```
mean(x, ...)
```

```
## Default S3 method:
```

```
mean(x, trim = 0, na.rm = FALSE, ...)
```

Arguments

- x** An R object. Currently there are methods for numeric/logical vectors and [date](#), [date-time](#) and [time interval](#) objects. Complex vectors are allowed for `trim = 0`, only.
- trim** the fraction (0 to 0.5) of observations to be trimmed from each end of `x` before the mean is computed. Values of `trim` outside that range are taken as the nearest endpoint.
- na.rm** a logical value indicating whether NA values should be stripped before the computation proceeds.
- ...** further arguments passed to or from other methods.

Figure 1: A portion of the output of `help(mean)`.

1 Objects in R

2 R is an **object oriented language**. During a given R session, one has any
3 number of objects in the current **environment**. These objects will include
4 vectors, matrices, lists, data frames, user-defined functions, and more.

5 The function `str()` provides useful information regarding any object.

6 Use `objects()` to see the current list of objects. The function `rm()` will
7 remove an object.

8 When exiting R using the `q()` function, the user is given the opportunity
9 to save all current objects; these will be reloaded upon restarting R in that
10 directory. See the functions `save()` and `load()` for storing selected R
11 objects.

- 1 **Exercise:** Describe a situation in which the `save()` function would be
2 useful. Can a single call to `save()` be used with multiple objects?

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11 _____

12 _____

1 **Vectors**

- 2 **Vectors** are the fundamental object in R and come in four basic types:

- 3 1. logical
- 4 2. integer
- 5 3. double
- 6 4. character

- 7 The function `c()` creates a vector.

- 8 The function `typeof()` will return the type of any vector.

- 9 There are **logical test functions** `is.logical()`, `is.integer()`,
10 `is.double()`, and `is.character()`.

- 1 Consider the following example:

```
> logex = c(TRUE, TRUE, FALSE) # These two lines define
> logex = c(T, T, F)           # the same logical vector
> typeof(logex)
[1] "logical"
```

```
> is.logical(logex)
[1] TRUE
```

```
> is.integer(logex)
[1] FALSE
```

```
> length(logex)
[1] 3
```

- 1 Integer vectors require the use of the L suffix:

```
> intex = c(4L, 2L, 6L, 1L)
> typeof(intex)
[1] "integer"
```

```
> is.double(intex)
[1] FALSE
```

```
> doubex = c(4, 2, 6, 1)
> typeof(doubex)
[1] "double"
```

```
> is.integer(doubex)
[1] FALSE
```

- 1 Consider a simple example of a character vector:

```
> charex = c("Hello", "World")
```

- 2 and note the behavior of `is.numeric()`:

```
> is.numeric(logex)
[1] FALSE
```

```
> is.numeric(intex)
[1] TRUE
```

```
> is.numeric(doubex)
[1] TRUE
```

```
> is.numeric(charex)
[1] FALSE
```

1 Coercion

- 2 Vectors can only have elements of a single type. Consider the behavior of
- 3 the following example:

```
> mixex = c(1, 2, "Q")
> print(mixex)
[1] "1" "2" "Q"
```

```
> typeof(mixex)
[1] "character"
```

- 4 The vector `mixex` is “coerced” into type `character` in a natural way.
- 5 There are functions `as.logical()`, `as.integer()`, `as.double()`,
- 6 and `as.character()` that will force coercion when possible.

- 1 TRUE and FALSE will be coerced into 1 and 0, respectively, and vice-versa.

```
> ex1 = c(1, 0, T)
> print(ex1)
[1] 1 0 1
```

```
> typeof(ex1)
[1] "double"
```

```
> as.logical(c(1, 1, 0, 1))
[1] TRUE TRUE FALSE TRUE
```

- 2 Some functions will automatically coerce the vector:

```
> sum(c(T, T, T, F))
[1] 3
```

- 1 **Exercise:** Inspect `help(rnorm)`. Use this to approximate the probability
- 2 that a standard normal random variable exceeds 1.0.

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11 _____

12

1 Missing Values in R

- 2 The symbol NA is reserved by R to denote “missing” values (interpreted
3 broadly). Including an NA in a vector will not coerce the vector into a char-
4 acter type. (Do not wrap the NA in quotes.)

```
> is.numeric(c(45, 78, 12, NA))  
[1] TRUE
```

```
> is.na(c(45, 78, 12, NA))  
[1] FALSE FALSE FALSE TRUE
```

- 5 Want to test for missing values? The any() function is helpful:

```
> any(is.na(c(45, 78, 12, NA)))  
[1] TRUE
```

1 Creating Vectors

- 2 There are alternatives to using c() :

```
> 1:10  
[1] 1 2 3 4 5 6 7 8 9 10
```

```
> seq(1, 10, by=2)  
[1] 1 3 5 7 9
```

```
> seq(1, 20, length=5)  
[1] 1.00 5.75 10.50 15.25 20.00
```

- 3 There are functions logical(), integer(), double(), and character()
4 which initialize vectors of the specified length.

1 Subsetting Vectors

2 Single square brackets are used to extract entries of a vector:

```
> vectex = seq(1, 20, by=3)
> vectex[3]
[1] 7
```

```
> vectex[3:6]
[1] 7 10 13 16
```

```
> vectex[c(4, 7, 2)] # You can reorder entries
[1] 10 19 4
```

```
> vectex[-c(4, 7, 2)] # Exclude entries 4, 7, 2
[1] 1 7 13 16
```

1 **Exercise:** Run the following code, and describe what happened:

```
> x = rnorm(5)
> x[order(x)]
```

2 _____

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

1 Lists

- 2 **Lists** in R can consist of elements of various types, including vectors of
3 different lengths:

```
> listex = list(c(T,F,T), 1:5, 6)
> print(listex)
[[1]]
[1] TRUE FALSE TRUE

[[2]]
[1] 1 2 3 4 5

[[3]]
[1] 6
```

- 1 Using a single square bracket subsets a list into a new list, while using
2 double brackets pulls out a component of a list:

```
> listex[2]      # This is a list with a single component
[[1]]
[1] 1 2 3 4 5
```

```
> typeof(listex[2])
[1] "list"
```

```
> listex[[2]]    # This is an integer vector of length 5
[1] 1 2 3 4 5
```

```
> is.list(listex[[2]])
[1] FALSE
```

- 1 **Exercise:** Consider the R command `unlist(listex[2:3])`. Describe
2 the behavior of this function. What about `unlist(listex)`?

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11 _____

12 _____

- 1 Components of a list can be named, then referenced using the `$` operator:

```
> names(listex) = c("AA", "BB", "CC")  
> names(listex)  
[1] "AA" "BB" "CC"
```

```
> listex$AA  
[1] TRUE FALSE TRUE
```

```
> listex$CC  
[1] 6
```

- 2 The output of many of the more sophisticated statistical functions in R is a
3 named list of results from the procedure.

- 1 **Exercise:** Execute the following commands:

```
> x = rnorm(100)
> y = rnorm(100)
> holdlmout = lm(y~x)
```

- 2 What happened here? Inspect the form of holdlmout.

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

- 1 **Attributes**

- 2 The names seen above are a special case of an **attribute** of an R object. Any
- 3 R object can be given arbitrary attributes using the `attr()` function:

```
> x = seq(20, 1, by=-2)
> attr(x, "keyproperty") = "A reversed sequence"
> attr(x, "keyproperty")
[1] "A reversed sequence"
```

```
> attributes(x)
$keyproperty
[1] "A reversed sequence"
```

1 Factors

- 2 Categorical variables are called **factors** in R. The `factor()` function defines
3 a variable as a factor, and gives it an attribute consisting of the **levels** of that
4 factor, i.e., the possible values:

```
> finalgrade = factor(c("C", "B", "B", "C", "A"))  
> levels(finalgrade)  
[1] "A" "B" "C"
```

```
> finalgrade = factor(c("C", "B", "B", "C", "A"),  
+                      labels=c("C", "B", "A"), ordered=T)  
> levels(finalgrade)  
[1] "C" "B" "A"
```

- 1 **Exercise:** How does R internally represent a factor? Consider the following
2 example to see why you should be careful:

```
> as.integer(factor(c(0, 1, 2)))  
[1] 1 2 3
```

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

1 Matrices

- 2 A **matrix** in R is two-dimensional array of elements of the same type. The
3 function `matrix()` creates a matrix from a vector, constructing it column-
4 by-column (unless you set `byrow=TRUE`):

```
> matex = matrix(1:6, nrow=2)
```

```
> print(matex)
```

```
      [,1] [,2] [,3]
[1,]     1     3     5
[2,]     2     4     6
```

```
> c(nrow(matex), ncol(matex))
```

```
[1] 2 3
```

- 1 The functions `cbind()` and `rbind()` are often used to build matrices
2 from vectors and/or other matrices:

```
> rbind(1:3, 4:6, 7:9)
```

```
      [,1] [,2] [,3]
[1,]     1     2     3
[2,]     4     5     6
[3,]     7     8     9
```

```
> cbind(cbind(1:3, 4:6, 7:9), rbind(1:3, 4:6, 7:9))
```

```
      [,1] [,2] [,3] [,4] [,5] [,6]
[1,]     1     4     7     1     2     3
[2,]     2     5     8     4     5     6
[3,]     3     6     9     7     8     9
```

- 1 **Exercise:** Suppose that n and m are defined, and each is a positive integer.
- 2 Write a single line of R code that creates a n by m matrix whose i^{th} row is
- 3 filled with i . Look at `help(rep)`.

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11 _____

12 _____

- 1 **Arrays**
- 2 Arrays are generalizations of matrices, and can be of any dimension. This
- 3 is a three-dimensional array:

```
> arrex = array(1:20, dim=c(2, 5, 2))  
> dim(arrex)  
[1] 2 5 2
```

```
> is.array(arrex)  
[1] TRUE
```

```
> is.matrix(arrex)  
[1] FALSE
```

```
> print(arrex)
, , 1

      [,1] [,2] [,3] [,4] [,5]
[1,]     1     3     5     7     9
[2,]     2     4     6     8    10

, , 2

      [,1] [,2] [,3] [,4] [,5]
[1,]    11    13    15    17    19
[2,]    12    14    16    18    20
```

1 Subsetting Matrices and Arrays

2 Square brackets are also used to subset matrices and arrays:

```
> matex[1,2]
[1] 3
```

```
> arrex[2,1,2]
[1] 12
```

```
> matex[,1:2]
      [,1] [,2]
[1,]     1     3
[2,]     2     4
```

- 1 R will simplify an object when it feels it's appropriate...

```
> matex[1,1:2]
[1] 1 3
```

```
> is.matrix(matex[1,1:2])
[1] FALSE
```

- 2 ...unless the `drop = FALSE` argument is provided:

```
> matex[1,1:2, drop=F]
      [,1] [,2]
[1,]     1     3
```

```
> is.matrix(matex[1,1:2, drop=F])
[1] TRUE
```

- 1 **Exercise:** Consider the following code:

```
> x = list("stats", 1:5, T, -10)
> dim(x) = c(2,2)
```

- 2 Describe the contents of `x`.

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9

1 Data Frames

2 **Data frames** in R are generalizations of matrices in that the columns of a
3 data frame can be of different types. They are the primary way in which
4 data sets are stored, given that most data we obtain is a mixture of numer-
5 ical information, text, and categorical variables (factors).

6 The function `data.frame()` creates the data frame:

```
> gradebook = data.frame(  
+   Name = c("Miguel", "Yotam", "Mary", "Jing"),  
+   HWScore = c(98, 87, 99, 78),  
+   ExamScore = c(92, 91, 87, 95),  
+   FinalGrade = c("A", "A-", "A", "B"),  
+   stringsAsFactors = FALSE)
```

1 **Exercise:** What happens if the argument `stringsAsFactors` is not set
2 to `FALSE`? How could one treat `Name` as text data, and `FinalGrade` as a
3 factor?

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11 _____

12 _____

- 1 R stores a data frame as a list of vectors. Like other lists, components can
- 2 be referenced using the `$` operator, and components can be easily added:

```
> mean(gradebook$HWScore)
[1] 90.5
```

```
> gradebook$SID = c("3782", "2390", "4623", "0376")
> print(gradebook)
```

	Name	HWScore	ExamScore	FinalGrade	SID
1	Miguel	98	92	A	3782
2	Yotam	87	91	A-	2390
3	Mary	99	87	A	4623
4	Jing	78	95	B	0376

1 Logical Subsetting

- 2 An alternative to subsetting with a vector of positions is to use a logical
- 3 vector, with `TRUE` in the entries to be retained, and `FALSE` in those posi-
- 4 tions to be excluded:

```
> x = c(45, 12, 78, 32, 89)
> x[c(T, T, F, F, T)]
[1] 45 12 89
```

- 5 A link is provided by the `which()` function. This handy function sim-
- 6 ply takes a logical vector and returns an integer vector consisting of the
- 7 positions of its `TRUE` entries:

```
> which(c(T, T, F, F, T))
[1] 1 2 5
```

- 1 **Exercise:** What happens when the length of the subsetting logical vector
- 2 does not match the length of the original vector?

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11 _____

12 _____

- 1 Use conditional statements to extract a portion of a data frame. The oper-
- 2 ator `==` indicates “equals” while `!=` is “not equals to.”

```
> gradebook[gradebook$ExamScore > 90,]
```

	Name	HWScore	ExamScore	FinalGrade	SID
1	Miguel	98	92	A	3782
2	Yotam	87	91	A-	2390
4	Jing	78	95	B	0376

```
> gradebook[gradebook$FinalGrade == "A",]
```

	Name	HWScore	ExamScore	FinalGrade	SID
1	Miguel	98	92	A	3782
3	Mary	99	87	A	4623